

Compiler Principle

Text Book: Modern Compiler Implementation in C

Author: Andrew W. Appel

2024 Spring&Summer

Content

1. INTRODUCTION
2. **LEXICAL ANALYSIS**
3. PARSING
4. ABSTRACT SYNTAX
5. SEMANTIC ANALYSIS
6. ACTIVATION RECORD
7. TRANSLATING INTO INTERMEDIATE CODE
8. BASIC BLOCKS AND TRACES
9. INSTRUCTION SELECTION
10. LIVENESS ANALYSIS
11. REGISTER ALLOCATION
13. GARBAGE COLLECTION
14. OBJECT-ORIENTED LANGUAGES
18. LOOP OPTIMIZATIONS

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



Thompson's Construction

Non-Deterministic Finite Automata



Subset Construction, DFA Minimization

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

2.4 Nondeterministic Finite Automata

Recall Deterministic Finite Automata

A finite automaton:

- A finite set of **states**;
- **Edges** lead from one state to another, and each edge is **labeled with a symbol** from the input symbol set/alphabet Σ ,
- One state is **the start state**,
- and certain of the states are distinguished as **final states**.
- In a *deterministic* finite automaton (DFA), **no two edges leaving from the same state** are labeled with the same symbol.

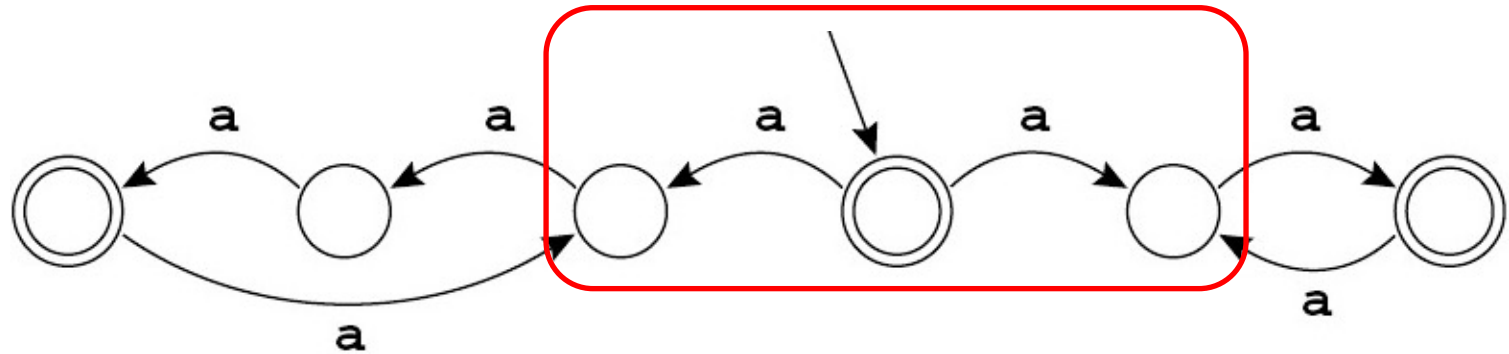
Nondeterministic Finite Automata

A **nondeterministic** finite automaton (NFA):

- may move to **multiple different states** from state s with input symbol a
- have **special edges** labeled with ϵ , which do not eat any symbol from input

Nondeterministic Finite Automata

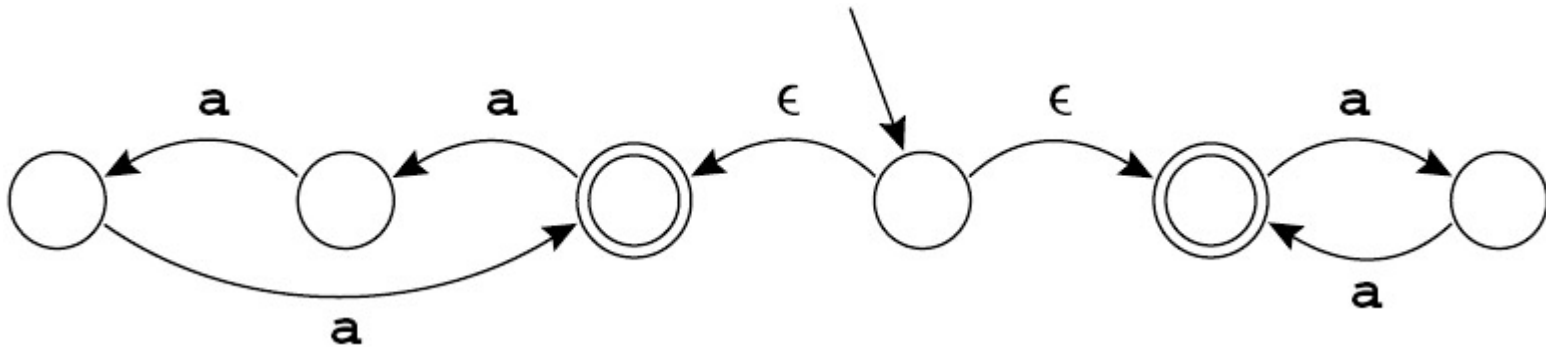
Here is an example of an NFA:



- **What language does the NFA recognize?**
 - All strings containing a multiple of two or three a's
- On the first transition, this machine must **choose which way to go.**
- It **must “guess” !!** And **always guess correctly!!**
 - Try & Rollback

Nondeterministic Finite Automata

Here is another NFA that accepts the same language:



Edges labeled with ϵ may be taken without using up a symbol from the input.

- The machine must choose which ϵ -edge to take !!
- A state with some ϵ -edges and edges labeled by symbols
 - Follow the corresponding symbol-labeled edge
 - Or to follow an ϵ -edge instead

How does an NFA accept a string?

- Recall DFA:

- Starting in the start state, for each character in the input string the automaton follows exactly **one edge** to get to the next state.
- The edge must be **labeled with the input character**.
- After making **n** transitions for an **n**-character string, if the automaton is in a **final state**, then it **accepts** the string.
- If it is not in a final state, or if at some point there was no appropriately labeled edge to follow, it **rejects**.

- An NFA accepts a string if there is any choice of paths that will lead to acceptance.

Nondeterministic Finite Automata

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression (**Static, Declarative**)



Easy

Thompson's Construction

NFA (**simulatable, quasi-executable**)



Deterministic Finite Automata



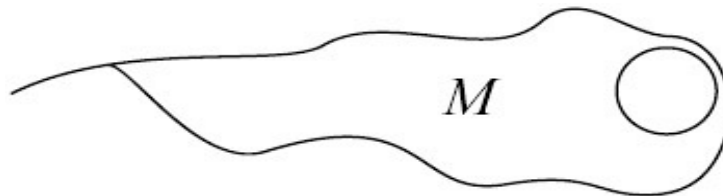
e.g., Table-Driven Implementation

Lexer

Converting a RE to an NFA

HOW TO CONVERT A REGULAR EXPRESSION TO AN NFA?

- Recall the definition of regular expressions.
- If we know:
 - How to convert the primitive regular expressions into primitive NFAs
 - How to simulate the three operations of regular expression construction for NFA construction
- Then, we can turn each regular expression into an NFA.
- In general, any regular expression M will have some NFA with a tail (start edge) and head (ending state):



Converting a RE to an NFA

The rules for translating regular expressions to non-deterministic finite automata

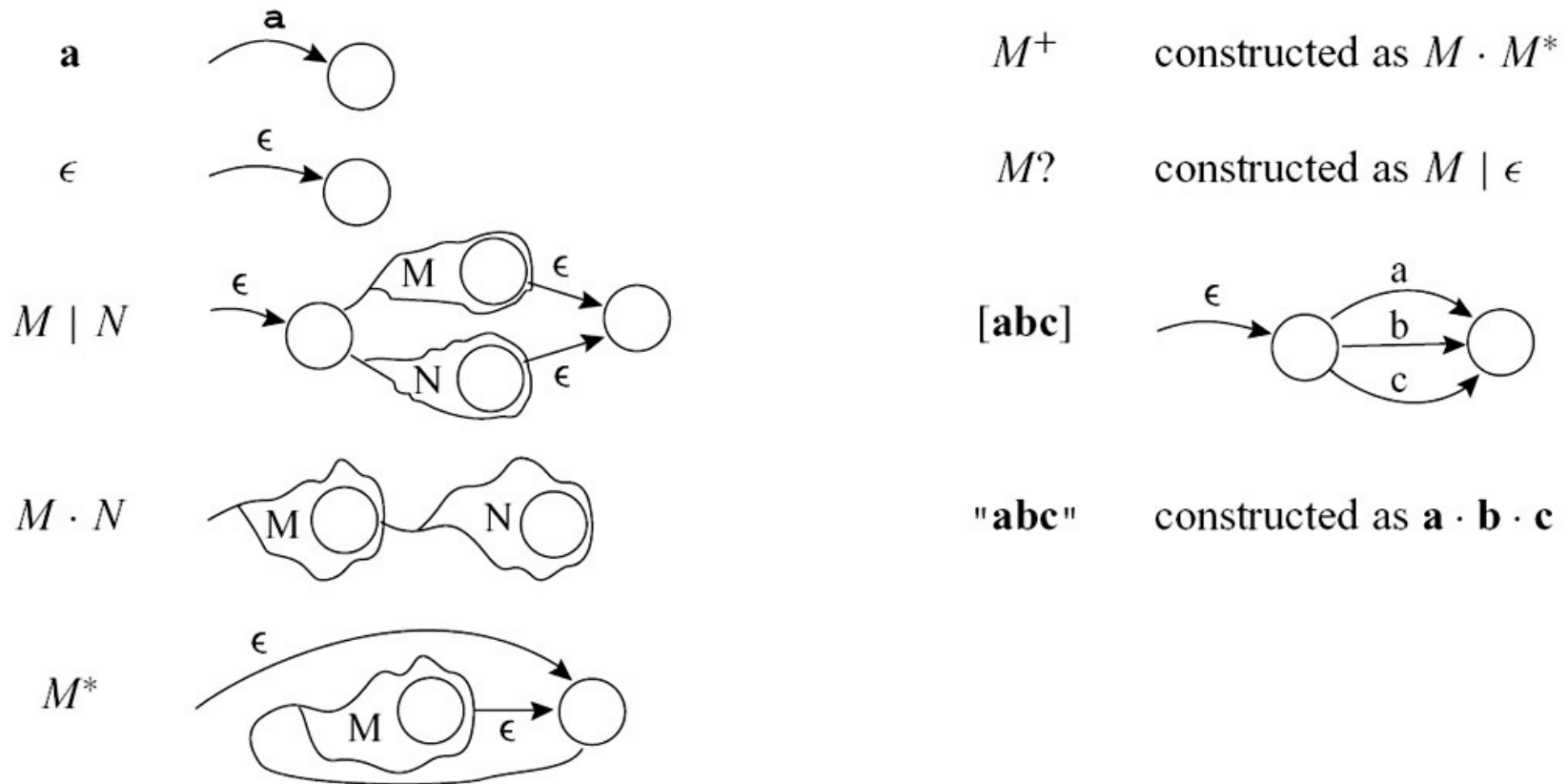


Figure 2.6: Translation of regular expressions to NFAs.

Converting a RE to an NFA

if	{return IF;}
[a-z][a-z0-9]*	{return ID;}
[0-9]+	{return NUM;}
([0-9]+"."[0-9]*) ([0-9]*"."[0-9]+)	{return REAL;}
("--"[a-z]*"\n") (" " "\n" "\t")+	{/*do nothing*/}
.	{ error();}

Figure 2.2: Regular expressions for some tokens

Converting a RE to an NFA

The result for the tokens IF, ID, NUM, and error

- After some merging of equivalent NFA states

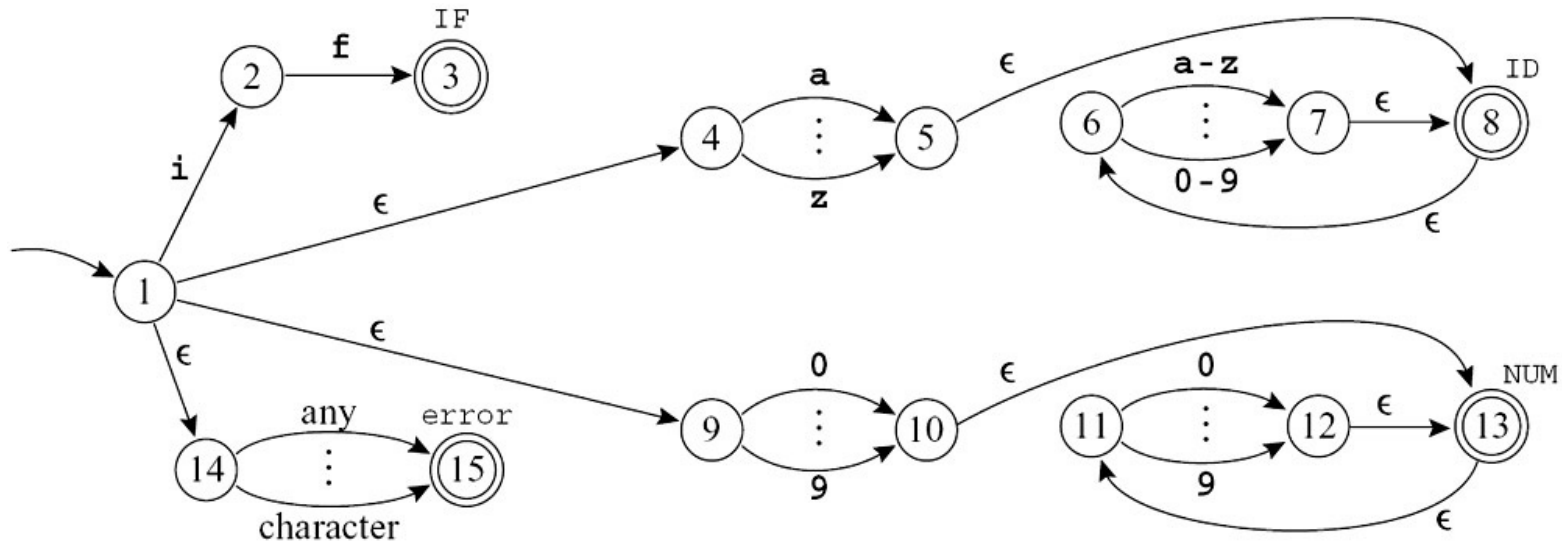


Figure 2.7: Four regular expressions translated to an NFA

- Each expression is translated to an NFA
- The “head” state marked final with a different token type
- The tails of all the expressions joined to a new start node

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



Non-Deterministic Finite Automata



? Subset Construction

Deterministic Finite Automata



Table-Driven Implementation

Lexer

Converting an NFA to an DFA - Idea

- Given state s and symbol a , we may need to guess the transition
- Idea: avoid guess. **But how?** **Simulate every possibility at once**
- For example, simulating the NFA of Figure 2.7 on *in*

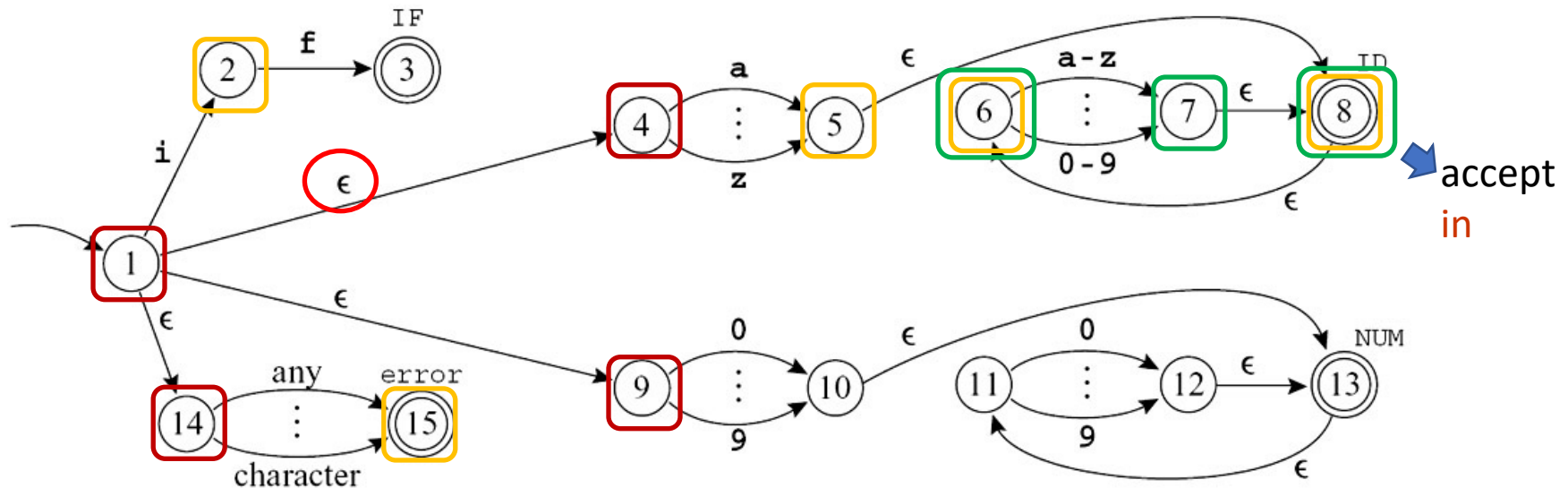


Figure 2.7: Four regular expressions translated to an NFA.

$\text{closure}(\{1\}) = \{1, 4, 9, 14\}$ // ϵ -closure of $\{1\}$

$\text{DFAedge}(\{1, 4, 9, 14\}, i) = \text{closure}(\{2, 5, 15\}) = \{2, 5, 6, 8, 15\}$

$\text{DFAedge}(\{2, 5, 6, 8, 15\}, n) = \text{closure}(\{7\}) = \{6, 7, 8\}$

Converting an NFA to an DFA - Idea

Writing the NFA simulation algorithm more formally:

- If the start state of the NFA is s_1 , the input string is $c_1, \dots, c_k$
- The algorithm is:

```

$$d \leftarrow \text{closure}(\{s_1\})$$

$$\text{for } i \leftarrow 1 \text{ to } k$$

$$d \leftarrow \text{DFAedge}(d, c_i)$$

```

- **Manipulating sets of states is expensive:**
 - Costly to do on every character in the source program
- Solution: do all the sets-of-states calculations in advance.
 - Start from $d_1 = \text{closure}(\{s_1\})$
 - In each d_i , for each symbol c_i , calculate $d_j = \text{DFAedge}(d_i, c_i)$.
 - Repeat until no new d can be found.
 - We can obtain all reachable d and the results of all possible $\text{DFAedge}(d_i, c_i)$.

Converting an NFA to an DFA - Idea

- We can construct an automaton by regarding each reachable d as a state.
 - **States:** the reachable d s of the NFA
 - **Transition function:** $DFAedge(d, c)$
 - **Alphabet:** the same as the NFA
 - **The start state:** d_1
 - **The final states:** the states (d s) that include at least a final state in the NFA
- This automaton is equivalent to the NFA. **Why?**
- This automaton is a deterministic automaton.
- This automaton is a finite automaton. **Why?**
 - Assume there are n states in the NFA
 - There are at most 2^n states in the new automation
- **We construct a DFA that is equivalent to the NFA.**

Converting an NFA to an DFA - Algorithm

Formally define **ϵ -closure** as follows

- For a set of states S , **closure(S)** is the set of states that can be reached from a state in S without consuming any of the input, that is, by going only through ϵ -edges.
- Mathematically, **closure(S) is the smallest set T** such that

$$T = S \cup \left(\bigcup_{s \in T} \text{edge}(s, \epsilon) \right)$$

- **edge(s, c)**: the set of all NFA states reachable by following a single edge with label c from state s .

Converting an NFA to an DFA - Algorithm

Calculating T by iteration

$$T = S \cup \left(\bigcup_{s \in T} \text{edge}(s, \epsilon) \right)$$

$T \leftarrow S$
repeat $T' \leftarrow T$
 $T \leftarrow T' \cup (\bigcup_{s \in T'} \text{edge}(s, \epsilon))$
until $T = T'$

Must the algorithm terminate?

- T can only grow before termination.
- There are only **a finite number of distinct states** in the NFA.
 T can at most be the set of all the NFA states.

Converting an NFA to an DFA - Algorithm

Define **DFAedge** as follows:

- Suppose a set $d = \{si, sk, sl\}$ of NFA states si ; sk ; sl .
- Starting in d and eating the input symbol c
- Reaching a new set of NFA states called set **DFAedge**(d, c)

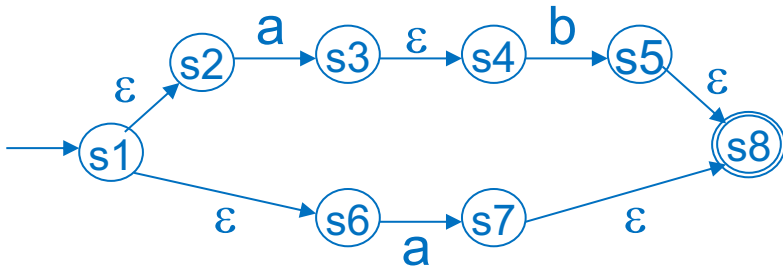
$$\mathbf{DFAedge}(d, c) = \mathbf{closure}(\bigcup_{s \in d} \mathbf{edge}(s, c))$$

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
           states[p]  $\leftarrow$  e
           trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

```

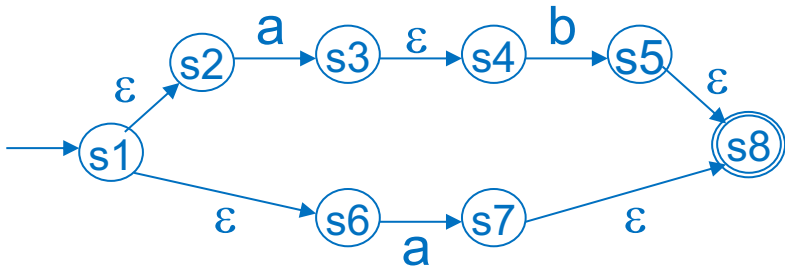
[illegible]

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
           states[p]  $\leftarrow$  e
           trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

```

[illegible]

Converting an NFA to an DFA

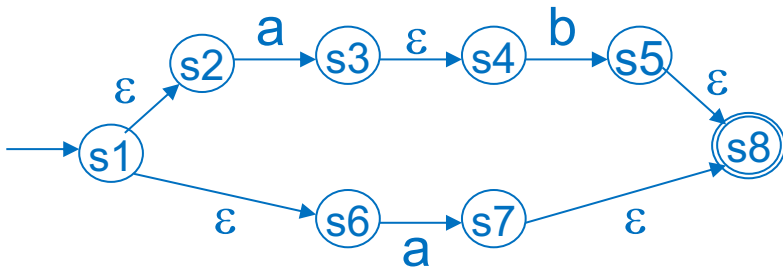
```
states[0] ← {};    states[1] ← closure( $\{s_1\}$ )
```

$$p \leftarrow 1; \quad j \leftarrow 0$$
while $j \leq p$ **foreach** $c \in \Sigma$
$$e \leftarrow \mathbf{DFAedge}(\text{states}[j], c)$$

if $e = \text{states}[i]$ for some $i \leq p$

then $\text{trans}[j, c] \leftarrow i$

else $p \leftarrow p + 1$

$$\text{states}[p] \leftarrow e$$
$$\text{trans}[j, c] \leftarrow p$$
$$j \leftarrow j + 1$$


states

trans

j	States		a	b
0	{}	0		
1	{s1, s2, s6}	1		
		2		
		3		

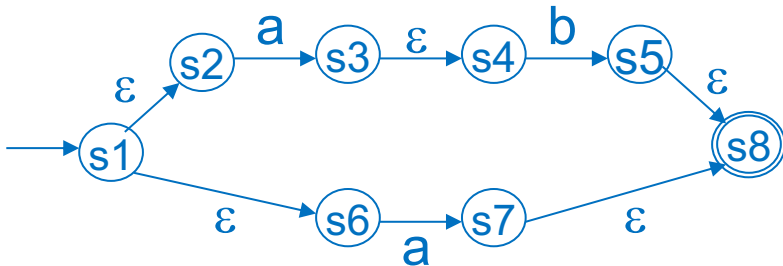
[illegible]

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
    foreach c  $\in$   $\Sigma$ 
        e  $\leftarrow$  DFAedge(states[j], c)
        if e = states[i] for some i  $\leq$  p
            then trans[j, c]  $\leftarrow$  i
        else p  $\leftarrow$  p + 1
            states[p]  $\leftarrow$  e
            trans[j, c]  $\leftarrow$  p
    j  $\leftarrow$  j + 1

```

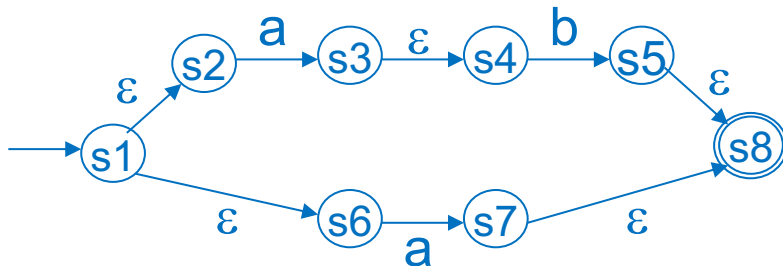
[illegible]

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
           states[p]  $\leftarrow$  e
           trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

```

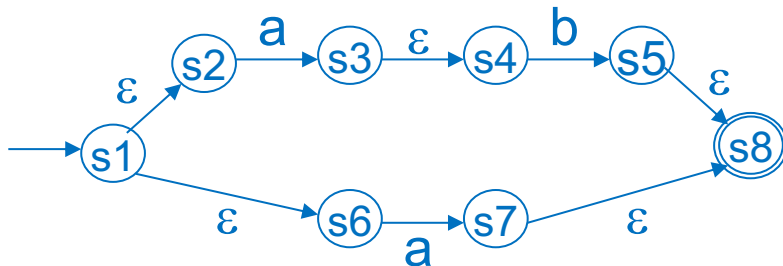
[illegible]

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
        states[p]  $\leftarrow$  e
        trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

```

[illegible]

Converting an NFA to an DFA

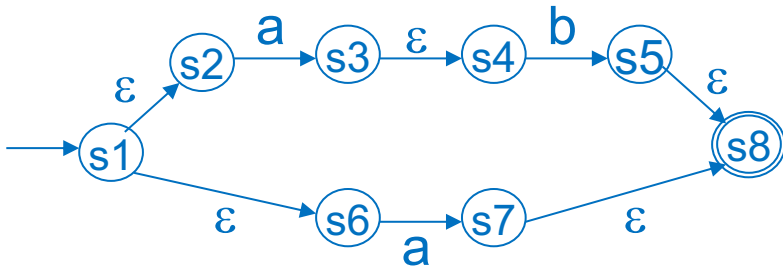
```
states[0] ← {};    states[1] ← closure( $\{s_1\}$ )
```

$$p \leftarrow 1; \quad j \leftarrow 0$$
while $j \leq p$ **foreach** $c \in \Sigma$
$$e \leftarrow \mathbf{DFAedge}(\text{states}[j], c)$$

if $e = \text{states}[i]$ for some $i \leq p$

then $\text{trans}[j, c] \leftarrow i$

else $p \leftarrow p + 1$

$$\text{states}[p] \leftarrow e$$
$$\text{trans}[j, c] \leftarrow p$$
$$j \leftarrow j + 1$$


states

trans

j	States		a	b
0	{}	0	0	0
1	{s1, s2, s6}	1		
		2		
		3		

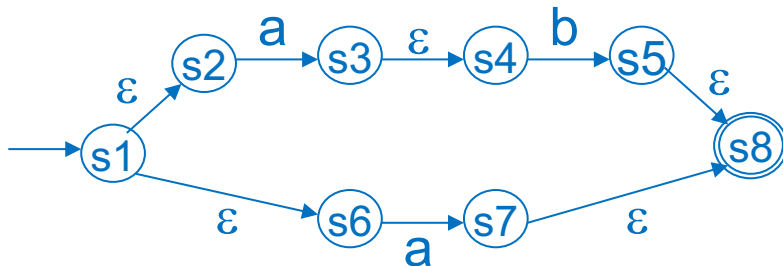
[illegible]

Converting an NFA to an DFA

```

states[0]  $\leftarrow$  {};      states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;      j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
      states[p]  $\leftarrow$  e
      trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

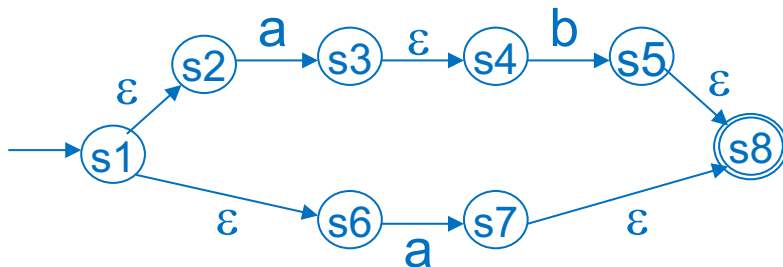
```

[illegible]

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
  foreach c ∈ Σ
    e ← DFAedge(states[j], c)
    if e = states[i] for some i ≤ p
      then trans[j, c] ← i
    else p ← p + 1
          states[p] ← e
          trans[j, c] ← p
  j ← j + 1
  
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3,s4,s7,s8}

trans

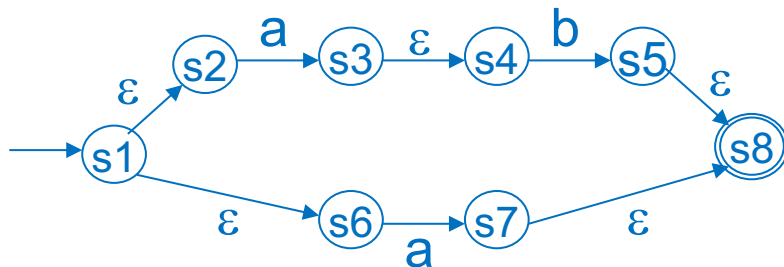
	a	b
0	0	0
1	2	0
2		
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2		

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
  foreach c ∈ Σ
    e ← DFAedge(states[j], c)
    if e = states[i] for some i ≤ p
      then trans[j, c] ← i
    else p ← p + 1
          states[p] ← e
          trans[j, c] ← p
  j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3, s4, s7, s8}

trans

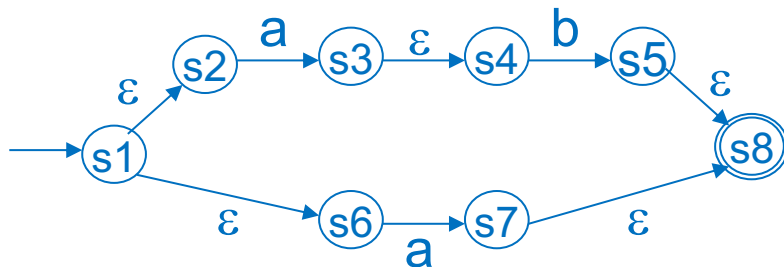
	a	b
0	0	0
1	2	0
2		
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3,s4,s7,s8}

trans

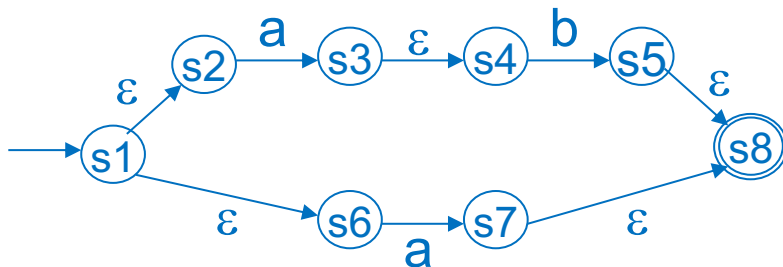
	a	b
0	0	0
1	2	0
2		
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}

Converting an NFA to an DFA

```

states[0] ← {};   states[1] ← closure({s1})
p ← 1;   j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3, s4, s7, s8}

trans

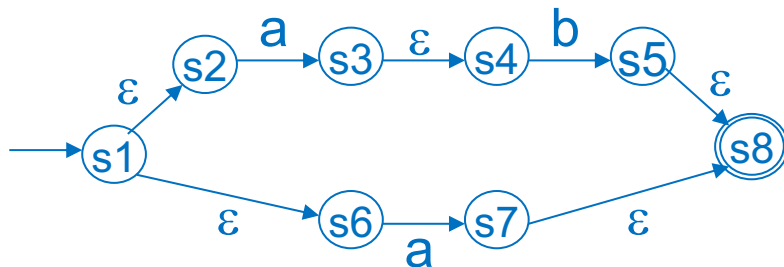
	a	b
0	0	0
1	2	0
2	0	
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3,s4,s7,s8}

trans

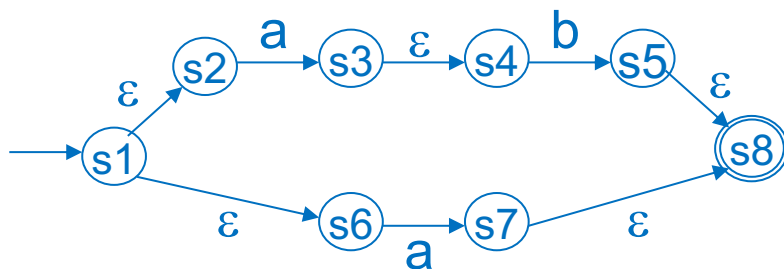
	a	b
0	0	0
1	2	0
2	0	
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3,s4,s7,s8}

trans

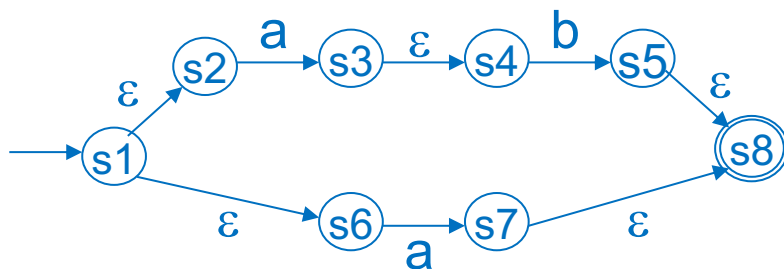
	a	b
0	0	0
1	2	0
2	0	
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	{s5, s8}

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3, s4, s7, s8}
3	{s5, s8}

trans

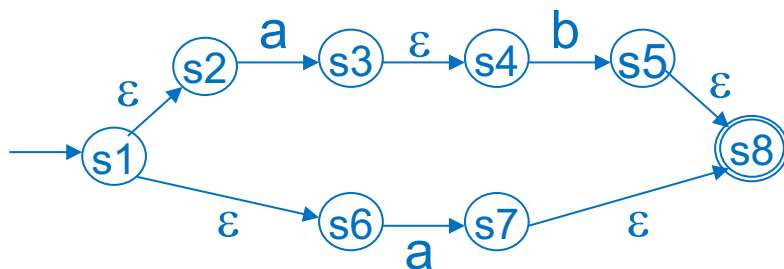
	a	b
0	0	0
1	2	0
2	0	3
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	{s5, s8}
3			

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
  foreach c ∈ Σ
    e ← DFAedge(states[j], c)
    if e = states[i] for some i ≤ p
      then trans[j, c] ← i
    else p ← p + 1
          states[p] ← e
          trans[j, c] ← p
  j ← j + 1
  
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3, s4, s7, s8}
3	{s5, s8}

trans

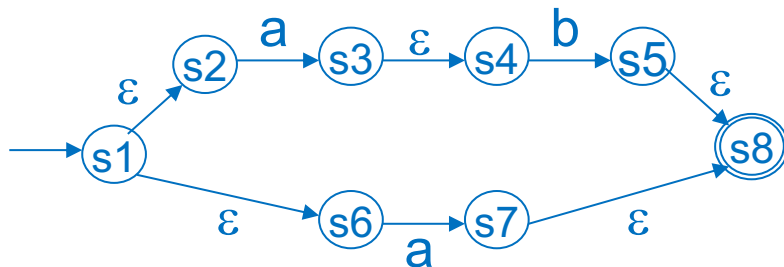
	a	b
0	0	0
1	2	0
2	0	3
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	{s5, s8}
3	3		

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
  foreach c ∈ Σ
    e ← DFAedge(states[j], c)
    if e = states[i] for some i ≤ p
      then trans[j, c] ← i
    else p ← p + 1
      states[p] ← e
      trans[j, c] ← p
  j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3, s4, s7, s8}
3	{s5, s8}

trans

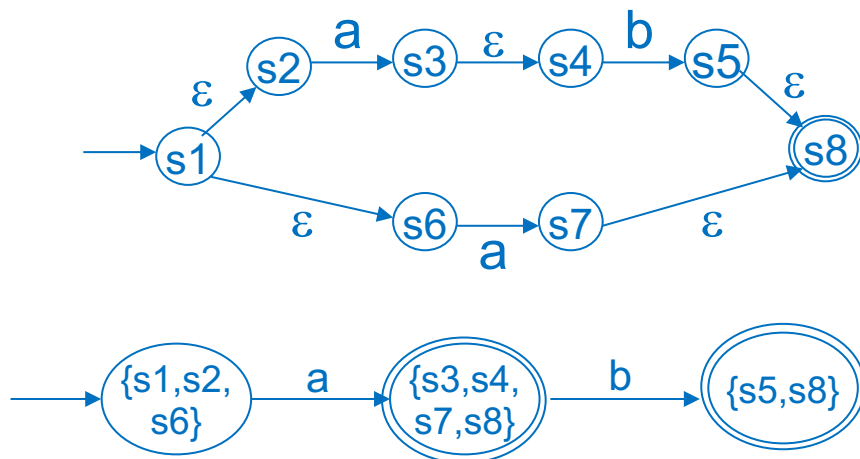
	a	b
0	0	0
1	2	0
2	0	3
3		

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	{s5, s8}
3	3	a	

Converting an NFA to an DFA

```

states[0] ← {};    states[1] ← closure({s1})
p ← 1;    j ← 0
while j ≤ p
    foreach c ∈ Σ
        e ← DFAedge(states[j], c)
        if e = states[i] for some i ≤ p
            then trans[j, c] ← i
        else p ← p + 1
            states[p] ← e
            trans[j, c] ← p
    j ← j + 1
    
```



states

j	States
0	{}
1	{s1, s2, s6}
2	{s3,s4,s7,s8}
3	{s5, s8}

trans

	a	b
0	0	0
1	2	0
2	0	3
3	0	0

p	j	c	DFAedge(states[j], c)
1	0	a	{}
1	0	b	{}
1	1	a	{s3, s4, s7, s8}
2	1	b	{}
2	2	a	{}
2	2	b	{s5, s8}
3	3	a	{}
3	3	b	{}
3	4	/	/

Converting an NFA to an DFA

- The algorithm does not visit unreachable states of the DFA
 - About n reachable states $\ll$ than 2^n states
 - Important to avoid an exponential blowup in the size of the DFA interpreter's transition tables
- A state d is final in the DFA if any NFA state in $\text{states}[d]$ is final in the NFA
- Labeling a state final is not enough
 - we must also say what token is recognized
- Several members of DFA state d are final in the NFA
 - Apply rule priority
- After the DFA is constructed
 - The "states" array may be discarded
 - The "trans" matrix is used for lexical analysis.

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



Thompson's Construction

Non-Deterministic Finite Automata



Subset Construction, **DFA Minimization**

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

DFA to Minimal DFA

Merging the NFAs of IF, ID, NUM, and error

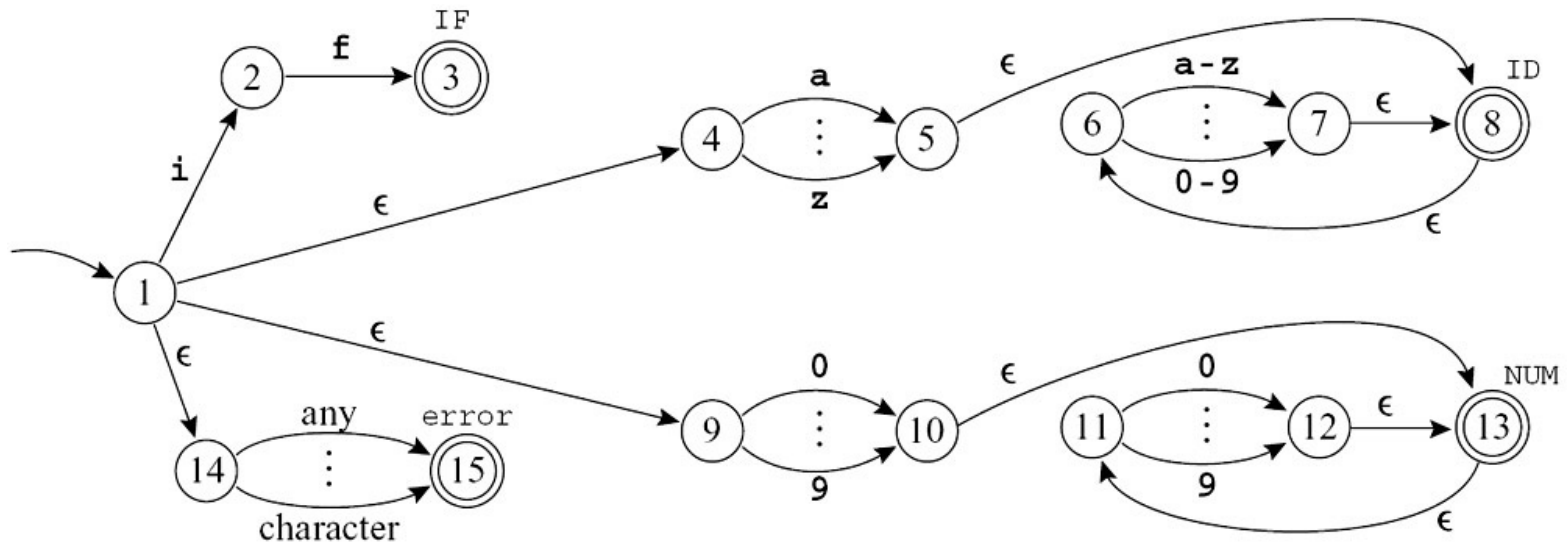


Figure 2.7: Four regular expressions translated to an NFA

DFA to Minimal DFA

Applying the DFA construction algorithm to the NFA of Figure 2.7 gives the automaton in Figure 2.8

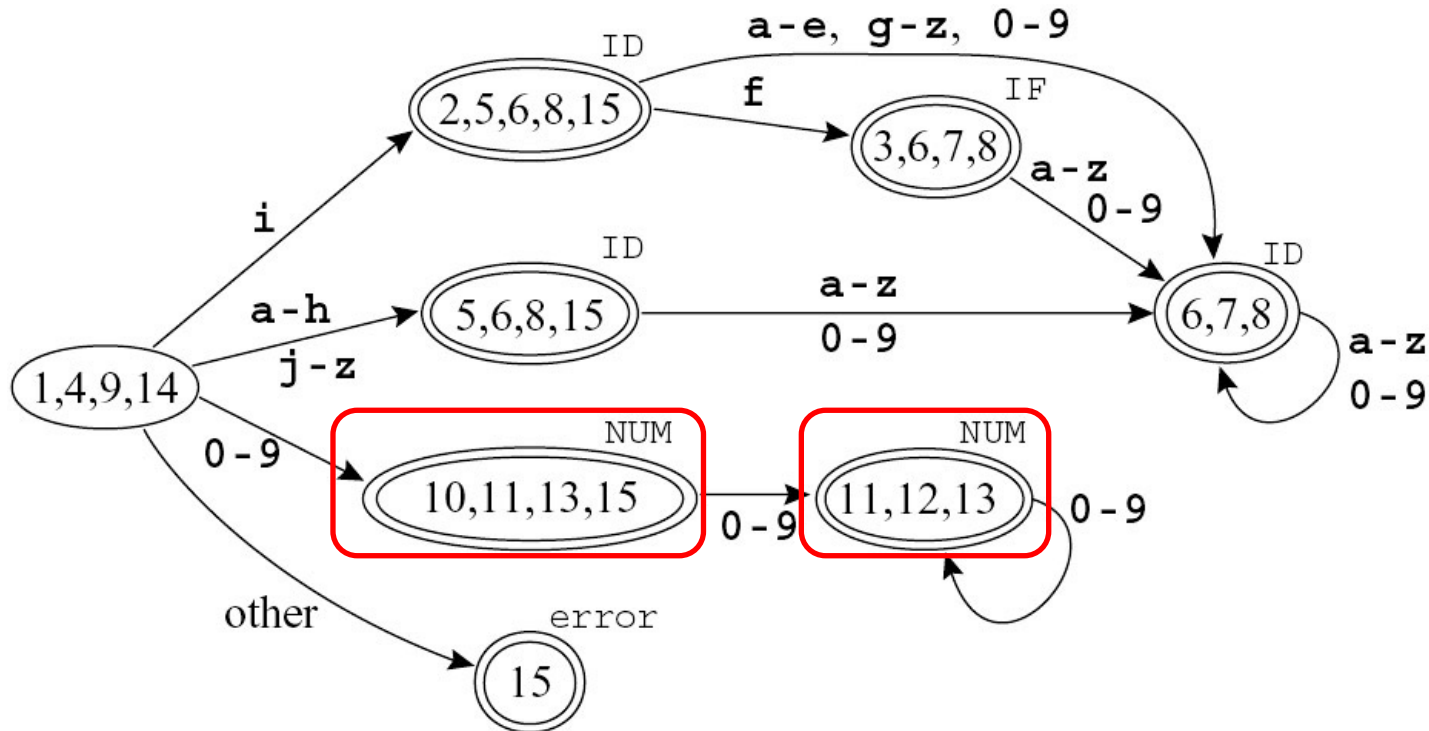


Figure 2.8: NFA converted to DFA

- **The automaton is suboptimal**
 - It is not the smallest one that recognizes the same language

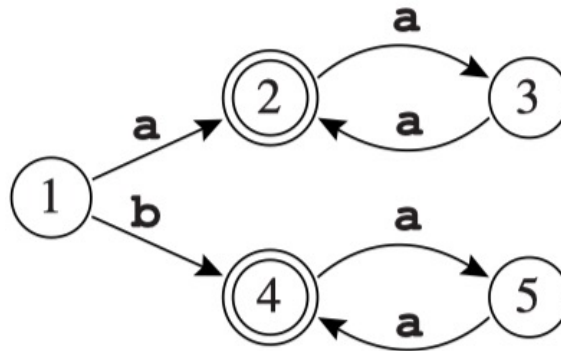
DFA to Minimal DFA

- Can we convert a DFA into a minimal DFA?
 - The key is to **find and merge equivalent states**.
- Two states $s1$ and $s2$ are **equivalent**
 - The machine starting in $s1$ accepts a string σ if and only if starting in $s2$ it accepts σ .
 - E.g., in Figure 2.8, $\{5,6,8,15\}$ and $\{6,7,8\}$, and $\{10,11,13,15\}$ and $\{11,12,13\}$
- In an automaton with two equivalent states $s1$ and $s2$
 - **Make all of $s2$'s incoming edges point to $s1$ instead and delete $s2$.**
 - E.g., delete $\{10, 11, 13, 15\}$ in Figure 2.8

DFA to Minimal DFA

How can we find equivalent states?

- Certainly, $s1$ and $s2$ are equivalent if they are:
 - both final or both nonfinal and,
 - for any symbol c , $\text{trans}[s1, c] = \text{trans}[s2, c]$;
- But this condition is **not sufficiently general**; consider the automaton



Here, states 2 and 4 are equivalent, but $\text{trans}[2, a] \neq \text{trans}[4, a]$.

DFA to Minimal DFA

- How to find equivalent states?
- **Distinguishable states:**
 - We say that string x **distinguishes** state s from state t if exactly one of the states reached from s and t by following the path with label x is a final state.
 - State s is **distinguishable** from state t if there is some string that distinguishes them.

DFA to Minimal DFA - Algorithm

DFA Minimization Algorithm (from the Dragon book)

- Split distinguishable states into different groups (iteratively)
 - Base case: ϵ distinguishes any final state (accepting state) from any non-final state (non-accepting state)
 - Inductive step: If states s and t are distinguishable, and there are edges with label a from state s' to s and state t' to t , s' and t' are also distinguishable.
- If two states are indistinguishable, they are equivalent.
 - All dead states are equivalent
- Choose a representative from each group and rebuild the DFA

DFA to Minimal DFA - Algorithm

DFA Minimization Algorithm

1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

 partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

 replace G in Π_{new} by the set of all subgroups formed;

}

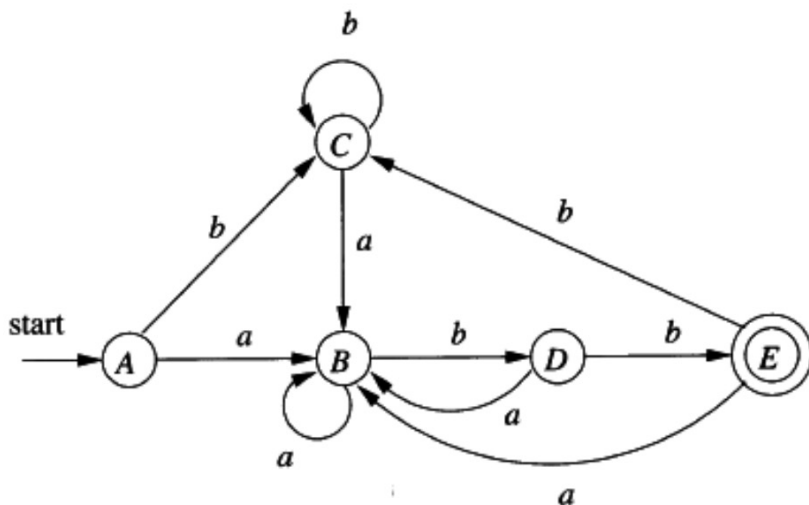
3. If $\Pi_{new} = \Pi$, let $\Pi_{final} = \Pi$ and continue. Otherwise, repeat step 2 with $\Pi = \Pi_{new}$

DFA to Minimal DFA - Algorithm

4. Choose one state in each group of Π_{final} as the representative for that group. The representatives will be the states of the minimum-state DFA D' .
 - The start state of D' is the representative of the group containing the start state of the original DFA.
 - The final states of D' are the representatives of those groups that contain a final state of D . (Such groups only contain final states).
 - Let s = the representative of G in Π_{final} , and let the transition of the original DFA from s on input a be to state t . Let r = the representative of t 's group H . Then in D' , there is a transition from s to r on input a .

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$		



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

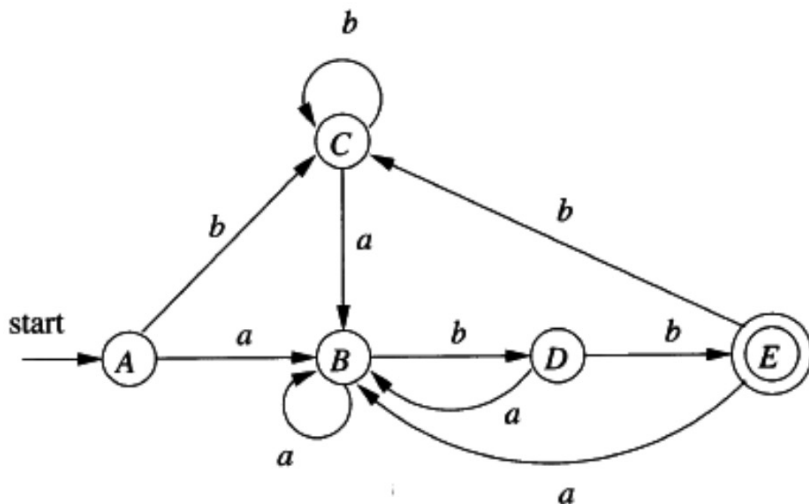
for (each group G in Π) {

partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

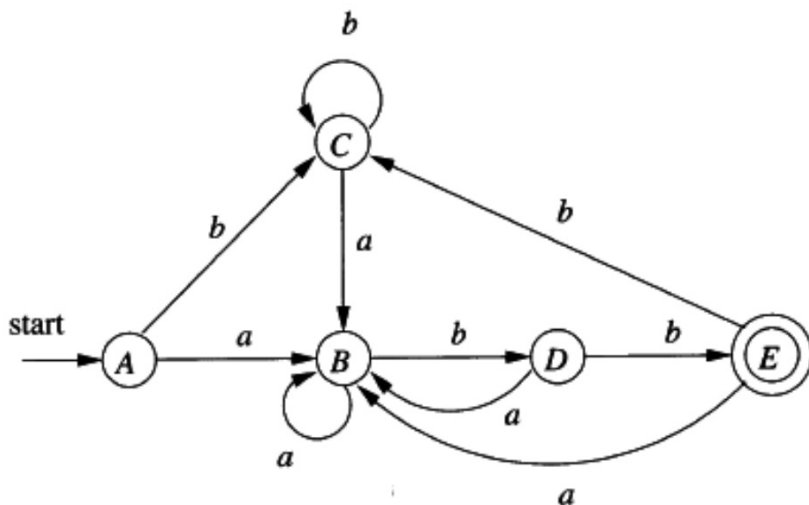
}

DFA to Minimal DFA - Example

[illegible][illegible]

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

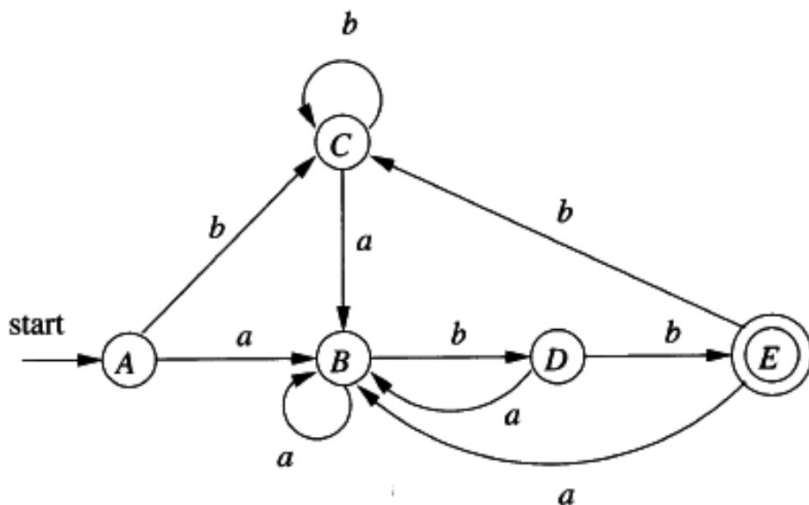
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

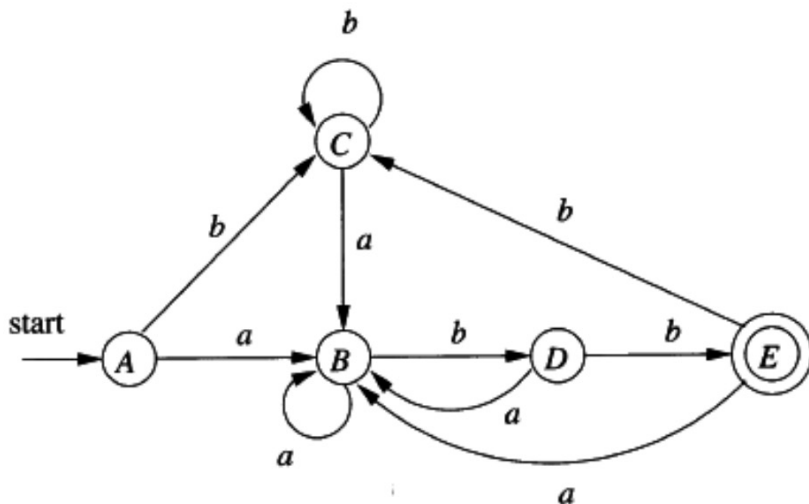
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

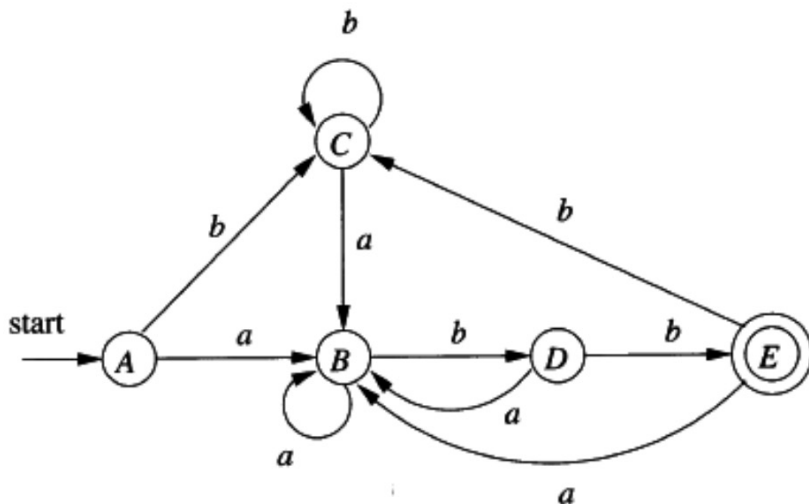
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$			



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

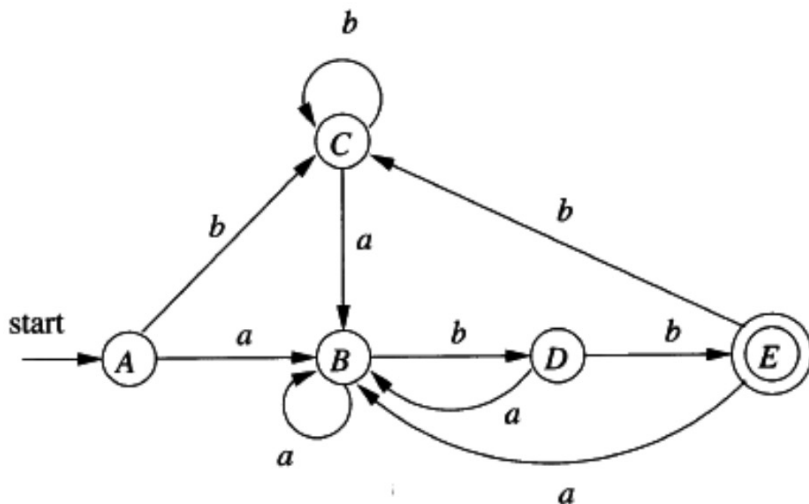
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$		



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

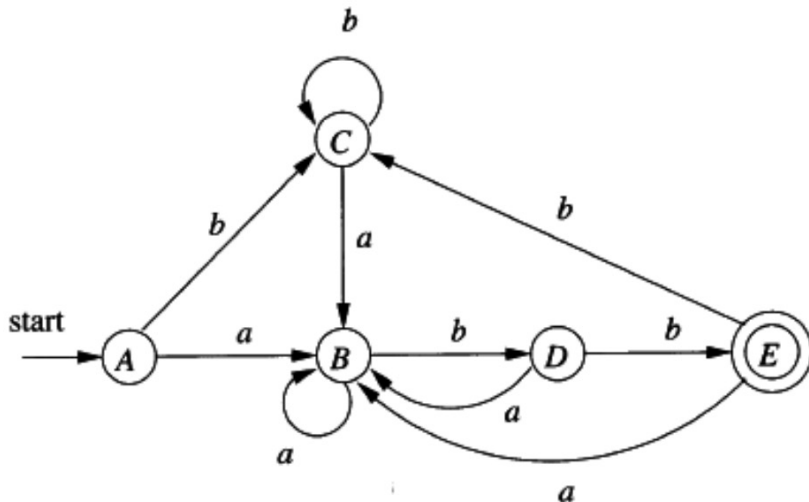
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

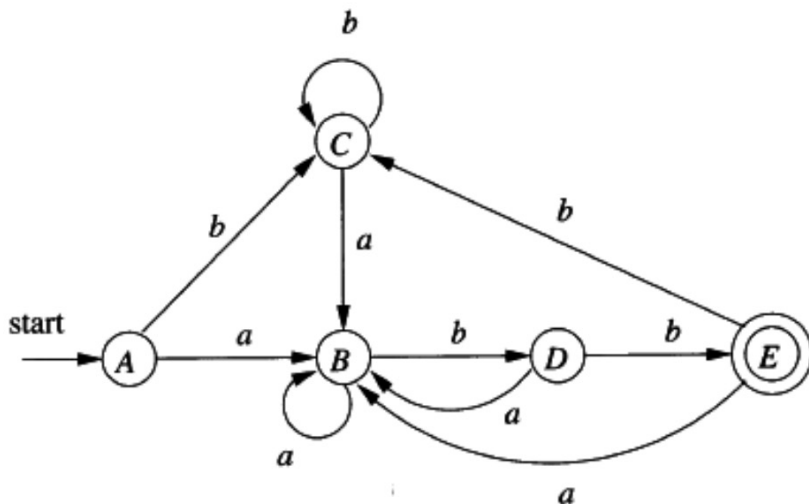
Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action



3. If $\Pi_{new} = \Pi$, let $\Pi_{final} = \Pi$ and continue. Otherwise, repeat step 2 with $\Pi = \Pi_{new}$

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$			



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

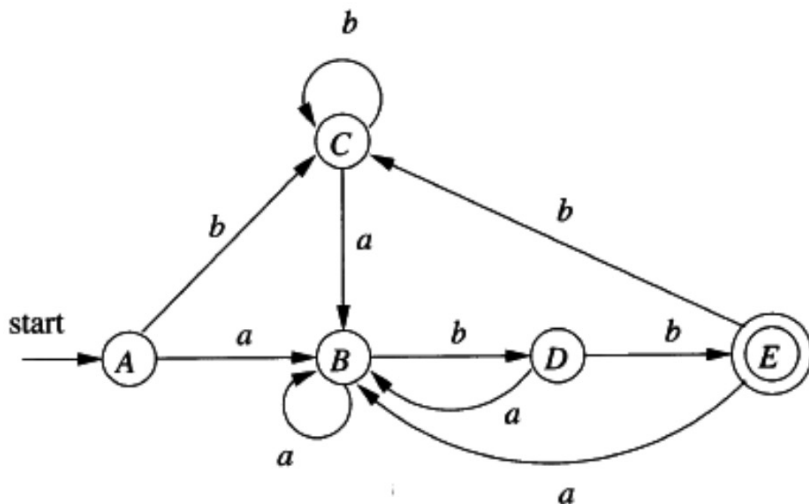
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

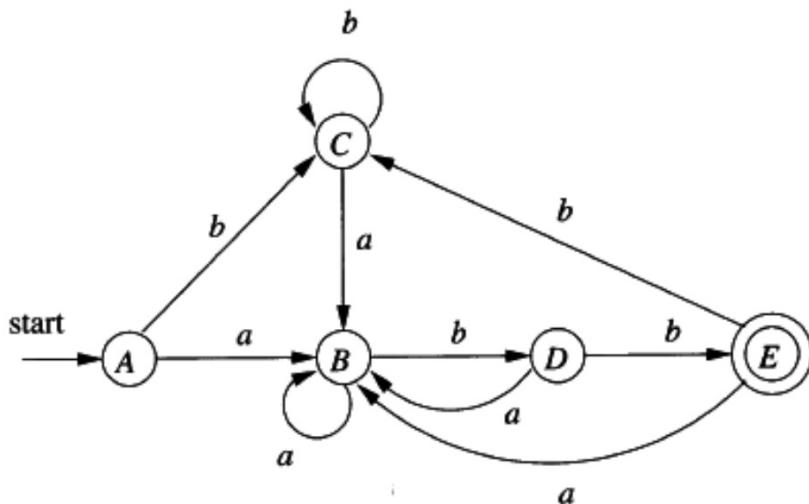
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

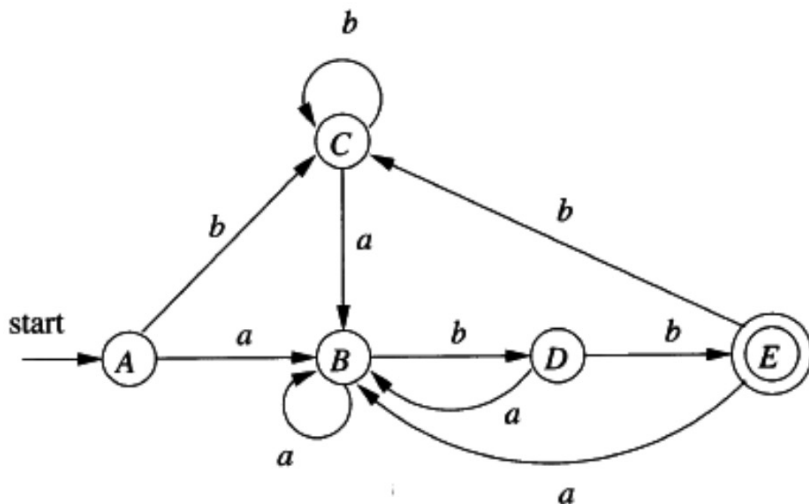
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

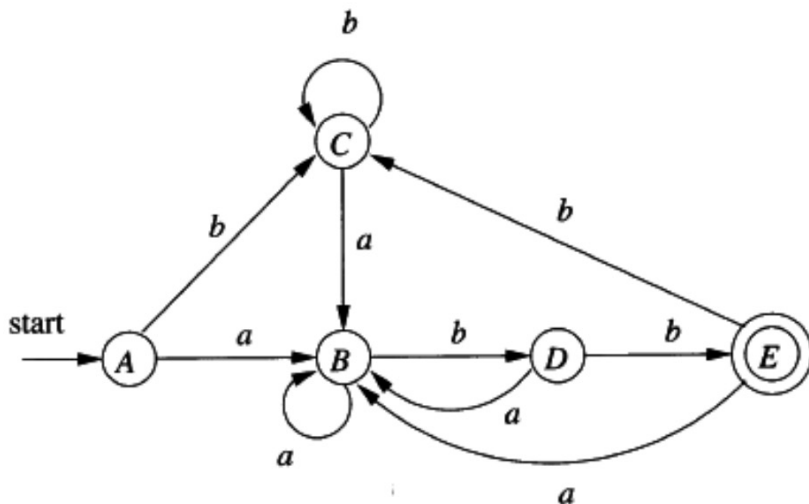
Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	



1. Start with an initial partition $\Pi = \{S - F, F\}$
2. Apply the following procedure
 let $\Pi_{new} = \Pi$
 for (each group G in Π) {
 partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;
 replace G in Π_{new} by the set of all subgroups formed;
 }

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	$\{A, B, C\} \Rightarrow \{A, C\}, \{B\}$

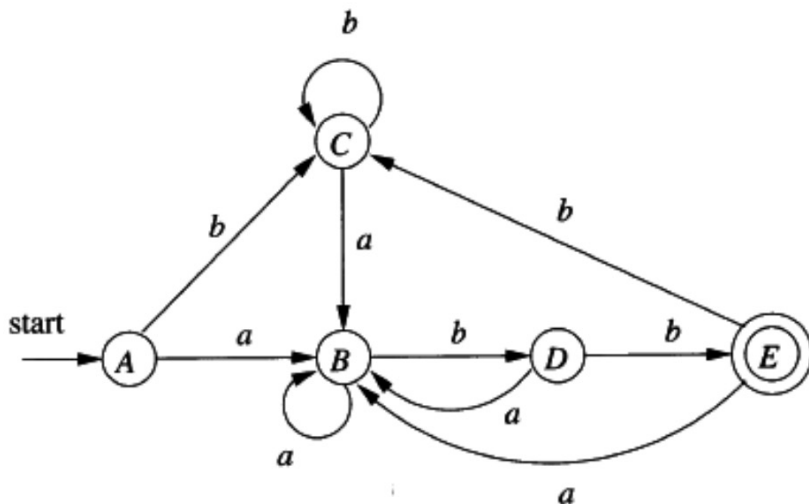


1. Start with an initial partition $\Pi = \{S - F, F\}$
2. Apply the following procedure

let $\Pi_{new} = \Pi$
 for (each group G in Π) {
 partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;
 replace G in Π_{new} by the set of all subgroups formed;
 }

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	$\{A, B, C\} \Rightarrow \{A, C\}, \{B\}$
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$			



1. Start with an initial partition $\Pi = \{S - F, F\}$

2. Apply the following procedure

let $\Pi_{new} = \Pi$

for (each group G in Π) {

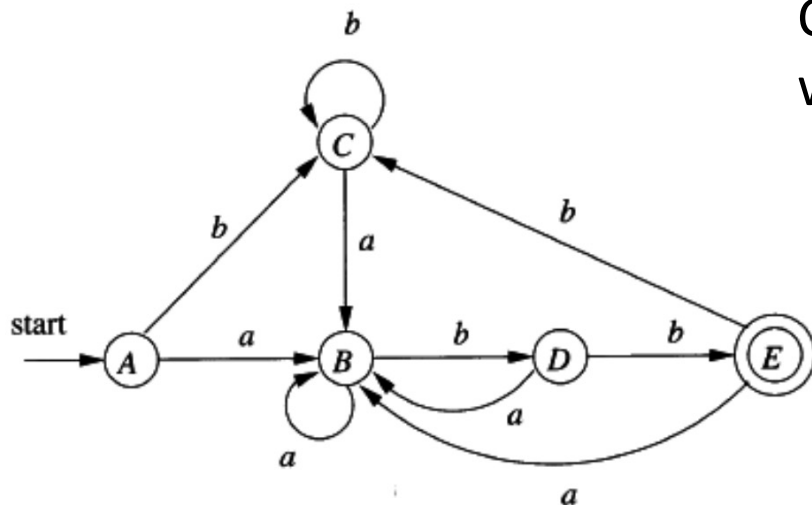
partition G into subgroups such that two states s and t are in the same subgroup iff for all input symbols a , states s and t have transitions on a to states in the same group of Π ;

replace G in Π_{new} by the set of all subgroups formed;

}

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	$\{A, B, C\} \Rightarrow \{A, C\}, \{B\}$
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{D\}$ or $\{E\}$	a / b	No action
$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{A, C\}$	a / b	No action

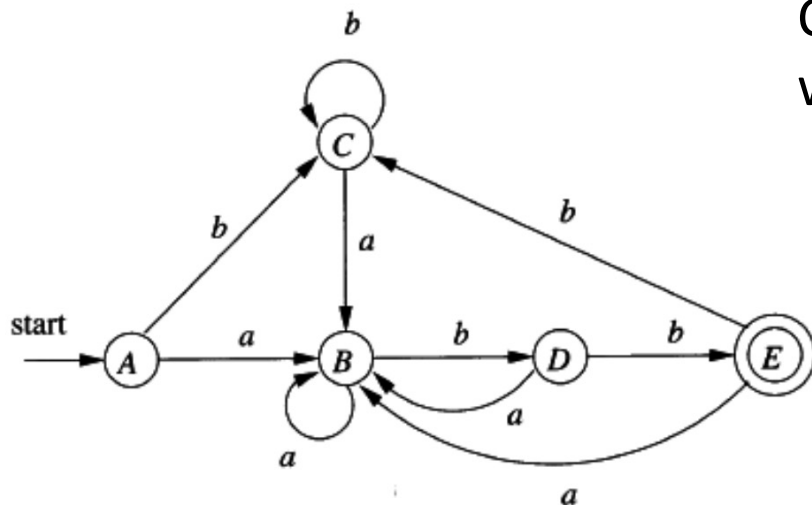


Choose A, B, D, E as the representatives,
we can get the minimal-state DFA:

State	a	b
A		
B		
D		
E		

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	$\{A, B, C\} \Rightarrow \{A, C\}, \{B\}$
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{D\}$ or $\{E\}$	a / b	No action
$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{A, C\}$	a / b	No action

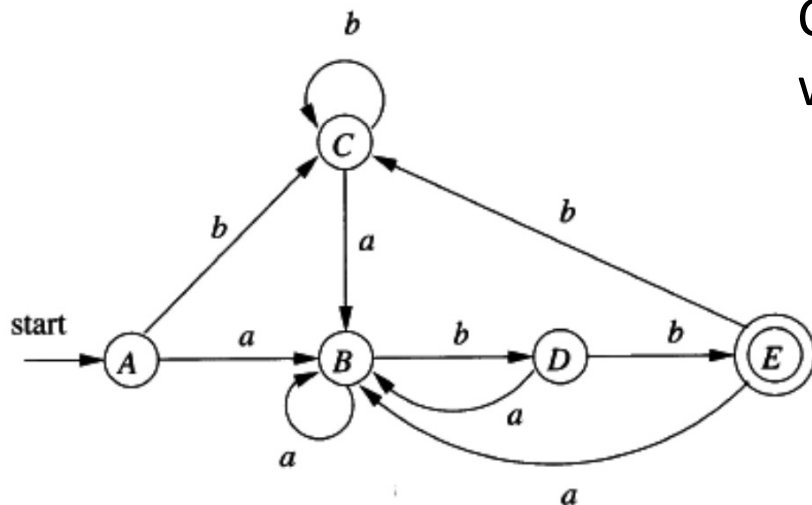


Choose A, B, D, E as the representatives,
we can get the minimal-state DFA:

State	a	b
A	B	A
B	B	D
D	B	E
E	B	A

DFA to Minimal DFA - Example

Π	Π_{new}	Group	Symbol	Action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	a	No action
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C, D\}, \{E\}\}$	$\{A, B, C, D\}$	b	$\{A, B, C, D\} \Rightarrow \{A, B, C\}, \{D\}$
$\{\{A, B, C, D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{E\}$	a / b	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	a	No action
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{A, B, C\}$	b	$\{A, B, C\} \Rightarrow \{A, C\}, \{B\}$
$\{\{A, B, C\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{D\}$ or $\{E\}$	a / b	No action
$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$	$\{A, C\}$	a / b	No action

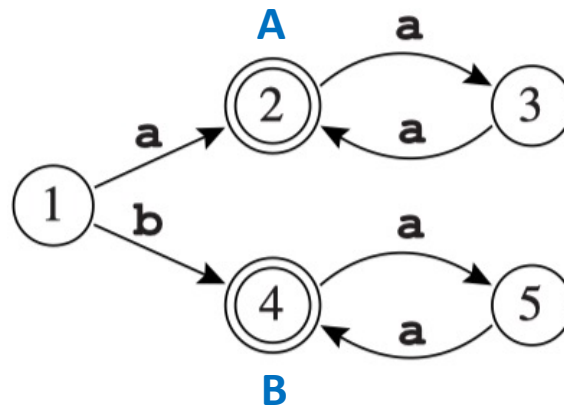


Choose A, B, D, E as the representatives,
we can get the minimal-state DFA:

State	a	b
$\{A, C\}$	$\{B\}$	$\{A, C\}$
$\{B\}$	$\{B\}$	$\{D\}$
$\{D\}$	$\{B\}$	$\{E\}$
$\{E\}$	$\{B\}$	$\{A, C\}$

State Minimization in Lexical Analyzers

- The final states in a lexical analyzer can correspond to different tokens.
 - They are not equivalent.



- To apply the state minimization procedure to the DFA generated for a lexical analyzer:
 - Initial Partition $\Pi = \{S-F, F_1, \dots, F_k\}$
 - F_i : all the states that recognize a particular token

2.5 Lex: A Lexical Analyzer Generator

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



Thompson's Construction

Non-Deterministic Finite Automata



Subset Construction, DFA Minimization

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

Lex: A Lexical Analyzer Generator

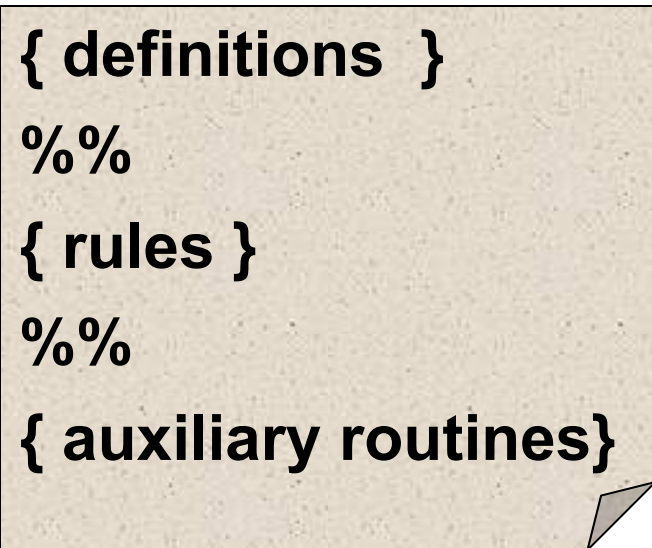
- **DFA construction:**
 - A mechanical task easily performed by computer
- **lexical-analyzer generator :**
 - Automatically translate regular expressions into a DFA
- **Lex:**
 - A lexical analyzer generator that produces a C program from a lexical specification.

Lex: A Lexical Analyzer Generator

- The lexical analyzer output by **Lex** contains C source code defining a procedure *yylex*
 - operates like a *getToken* procedure
 - a table-driven implementation of a DFA corresponding to the regular expressions of the input file
- The most popular version of Lex is called flex (Fast Lex)

Lex: A Lexical Analyzer Generator

- The format of a Lex input file (*a.l*)



{ definitions }
%%
{ rules }
%%
{ auxiliary routines }

```
%{  
#include <stdlib.h>  
#include <stdio.h>  
int count = 0;  
%}  
digit [0-9]  
number {digit}+  
%%  
{ number } { int n = atoi (yytext);  
              printf("%x", n);  
              if (n > 9) count ++; }  
%%  
main( )  
{ yylex ();  
  fprintf(stderr, "number of replacements =  
%d", count);  
  return 0;  
}
```


Lex: A Lexical Analyzer Generator

1. The first section: *definitions*

- The definition section occurs before the first `%%`.
- Between the `%{...%}`: `includes` and `declarations` that may be used by the C code in the remainder of the file
- After `%{...%}`: regular-expression abbreviations and state declarations.
 - `{digit}`

```
%{  
#include <stdlib.h>  
#include <stdio.h>  
int count = 0;  
%}  
digit [0-9]  
number {digit}+
```

User Code

Lex: A Lexical Analyzer Generator

2. The second section: *rules*

- A rule contains a regular expression and an action.
- The actions are fragments of ordinary C code.
- When a regular expressions is matched, the corresponding action will be executed.
- `yytext`: the string matched by the RE
- `yyleng`: the length of the matched string

```
%%  
{ number } { int n = atoi (yytext);  
              printf ("%x", n);  
              if (n > 9) count ++; }  
%%
```

Lex: A Lexical Analyzer Generator

2. The third section: *auxiliary routines*

- Routines are called in the second section and not defined elsewhere.

```
main( )  
{ yylex ();  
  fprintf(stderr, "number of  
replacements = %d", count);  
  return 0;  
}
```

Read Section 2.5 and the manual of Lex for more details.

```

%{
/* a Lex program that changes all numbers from decimal to hexadecimal
notation, printing a summary statistic to stdeer
*/

#include <stdlib.h>
#include <stdio.h>
int count = 0;
}%
digit [0-9]
number {digit}+
%%
{ number } { int n = atoi (yytext);
              printf("%x", n);
              if (n > 9) count ++; }

%%
main( )
{ yylex ( );
  fprintf(stdeer, "number of replacements = %d", count);
  return 0;
}

```

Exercise

- 2.1, 2.2, 2.5 (a,b), 2.6

The End